Real-time Earthquake & Aftershock Detection Using Streaming Data

Proposed data model:

Data source:

Data sources include the United States Geological Survey (USGS) real-time earthquake GeoJSON feeds, which have constantly updated earthquake data. We used the USGS feeds for earthquakes that occurred in the last 30 days and last hour.

All_day:
https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_day.geojson
All_hour:
https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_hour.geojson

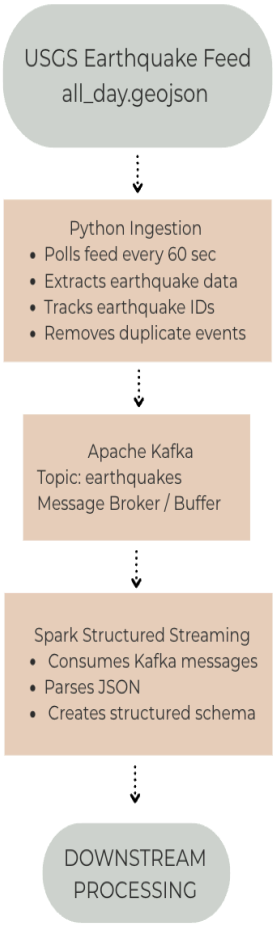
Data Type:

- This dataset is in the form of a GeoJSON stream that is provided by the USGS Earthquake Hazards Program.
- The data is provided as a continuously updated GeoJSON feed by the USGS Earthquake Hazards Program.
- An earthquake event is captured using a GeoJSON feature that contains attributes like:
    - ❖ Time
    - ❖ Latitude
    - ❖ Longitude
    - ❖ Depth
    - ❖ Magnitude
    - ❖ Place
    - ❖ Type of event.
- There might be missing or null attributes in the data obtained from this dataset, and there can also be outliers present.

Data Retrieval:

We developed a streaming ingestion pipeline that streams earthquake data in real-time from the USGS GeoJSON feed using Python. The ingestion service queries the feed at fixed intervals, picks out relevant earthquake information, and identifies unique earthquakes based on their ID numbers. The new events discovered through the process are streamed as JSON documents to an Apache Kafka topic called earthquakes. This Kafka topic is then consumed by Spark Structured Streaming which processes the data in structured form.

Notebook link:
https://colab.research.google.com/drive/1nD5LPUfKO_H0u8K2D96m7BcV1nHYAquC?usp=sharing



Data Storage:

The proposed system will use **MinIO** as the object storage layer for the earthquake data. MinIO is an S3-compatible object storage system that can store large volumes of data as objects and can be deployed locally using Docker. It is suitable for this project because it allows the system to store continuously generated earthquake data without requiring a cloud account during development. The stored data can later be migrated to cloud object storage such as Amazon S3 if required.

Delta Lake will be used as the table management layer on top of MinIO. It allows the stored earthquake data to be managed as reliable tables and works well with Apache Spark for both streaming and historical analytical workloads. This combination provides scalable storage while supporting the project's requirement for continuous processing and historical analysis.

Data Arrangement:

Bronze:

The Bronze layer stores raw earthquake data exactly as received from the USGS feed. It also includes metadata like the ingestion timestamp and USGS event ID. Keeping the raw data is important because USGS may later update details such as magnitude or location.

Silver:

The Silver layer contains cleaned and validated data. Duplicate records are removed using the event ID, and fields like time, location, depth, and magnitude are standardized. Missing values are handled properly, while unusual magnitudes are flagged instead of deleted. The data is partitioned by event date for faster analysis.

Gold:

The Gold layer stores aggregated information about mainshock–aftershock clusters. It includes details such as mainshock magnitude, number of aftershocks, geographic extent, and aftershock activity over time. This makes it suitable for dashboards and analytics without repeatedly processing raw data.

Data Preparation and Processing Pipeline:

The processing begins when Spark Structured Streaming gets continuous readings of earthquakes from the earthquakes topic in the Kafka. The incoming records will be stored in the Bronze layer in their unaltered form.

Data cleaning and validation follow before storing the data in the Silver layer. At this point, data cleaning involves removal of duplicate entries with respect to the USGS event id, standardization of timestamps, and treatment of invalid entries. Invalidation of unusual earthquake values is indicated at this stage.

Cleaned data will be used in spatiotemporal clustering in order to determine the possible mainshock-aftershock relationship. Data clustering will take into consideration factors like the magnitude of earthquakes and the proximity between them both geographically and temporally.

Clustered information will be stored together with other statistics like the number of aftershocks, geographical distribution, and the frequency of earthquake activities in the Gold layer.

Pipeline Flow:

Kafka → Spark Structured Streaming → Bronze → Cleaning & Validation → Silver → Aftershock Clustering → Statistics Calculation → Gold → Dashboard

Architecture and Design Decisions :

Choice	Selected	Alternative	Why Selected
Data Source	USGS GeoJSON Feed	Static dataset	Continuously updated earthquake data
Data Retrieval	Python Polling	Manual Download	Automatically retrieves new events
Messaging	Apache Kafka	Direct API → DB	Decouples ingestion and support streaming
Processing	Spark Structured Streaming	Batch Processing	Enables near real-time processing
Storage	MinIO + Delta Lake	Relational DB only	Supports scalable storage and analytics
Data Arrangement	Bronze → Silver → Gold	Single-layer storage	Separates raw, clean and enriched data
Data Format	GeoJSON --> JSON	CSV	Preserves USGS structure and geographic information
Planned Deployment	Docker + Kubernetes	Single Machine	Supports containerization and scalability

Alternative Architectures Considered :

Alternative	Why not selected
Direct API → Database	Tightly couples ingestion and storage and does not provide a decoupled streaming pipeline
Relational DB only	Less suitable for retaining raw data and supporting the Bronze-Silver-Gold lakehouse workflow.
MongoDB	Good for semi-structured data but less aligned with the planned Spark + lakehouse analytical workflow.
Cloud object storage	Suitable for production, but MinIO provides a simpler local development environment without cloud dependency.

Conclusion:

The project implements an earthquake data pipeline almost in real-time using the USGS earthquake feed. The earthquake data were collected and processed using Python poller, new earthquakes detected based on the earthquake IDs, and then the data were sent to the next stage using Apache Kafka. Then, Spark Structured Streaming was used to consume Kafka messages and process JSON data.

Concerning the storage part, Docker is deployed and the storage layer is designed using MinIO and Delta Lake, where the data are managed with the use of the Bronze-Silver-Gold architecture in order to store raw, cleaned and processed earthquake data.

Tracker Link: Project_Tracker_Group2

Research Papers

1. **Gardner, J. K., & Knopoff, L. (1974)** — *Is the Sequence of Earthquakes in Southern California, with Aftershocks Removed, Poissonian?*
   Relevant to **aftershock identification using spatial and temporal windows**.
   https://doi.org/10.1785/bssa0640051363?utm_.com
2. **Utsu, T., Ogata, Y., & Matsu'ura, R. S. (1995)** — *The Centenary of the Omori Formula for a Decay Law of Aftershock Activity.*
   Relevant to **aftershock decay over time / Omorilaw**.
   https://www.jstage.jst.go.jp/article/jpe1952/43/1/43_1_1/_article?utm_.com
3. **Parsons, T. (2002)** — *Global Omori Law Decay of Triggered Earthquakes: Large Aftershocks Outside the Classical Aftershock Zone.*
   Relevant to **temporal and spatial behavior of aftershocks**.
   https://pubs.usgs.gov/publication/70024161?utm_.com

White Papers / Technical References

- Apache Kafka Documentation — Relevant to event streaming and message-based ingestion. https://kafka.apache.org/43/getting-started/introduction/
- Apache Spark Structured Streaming Documentation — Relevant to continuous stream processing.https://spark.apache.org/streaming/?utm_.com
- Armbrust et al. (2020) — *Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores* — Relevant to the proposed storage architecture.
  https://www.vldb.org/pvldb/vol13/p3411-armbrust.pdf?utm_.com

Patents

- CN114298104A (2022) — *Earthquake Early Warning Method...* — Relevant to automated earthquake detection and warning.
  https://patents.google.com/patent/CN114298104A/en
- WO2019162858A1 (2019) — *Method and System to Generate an Earthquake Early Warning* — Relevant to earthquake early-warning systems.https://patents.google.com/patent/WO2019162858A1/en